Lecture 2. Data Visualisation

ggplot2: aesthetics, facets, geoms, stats, positions, coordinates, graphics for communication

Contents

2.1 Data visualisation with ggplot2 **2**
- 2.1.1 The mpg data frame . . . 2
- 2.1.2 Creating a ggplot . . . 2

2.2 Aesthetic mappings **3**

2.3 Facets **3**

2.4 Geometric objects **4**

2.5 Statistical transformations **5**

2.6 Position adjustments **6**

2.7 Coordinate systems **7**

2.8 The layered grammar of graphics **8**

2.9 Graphics for communication **8**
- 2.9.1 Labels . . . 8
- 2.9.2 Annotations . . . 9
- 2.9.3 Scales, legends and zooming . . . 9
- 2.9.4 Themes and saving . . . 10

2.1 Data visualisation with ggplot2

Motivation. ggplot2 builds every plot from one small set of parts.

Load the tidyverse (attaches `ggplot2`):

```
library(tidyverse)
```

2.1.1 The mpg data frame

Do cars with big engines use more fuel? `ggplot2::mpg` records 234 cars; `displ` is engine size (litres), `hwy` highway efficiency (miles per gallon).

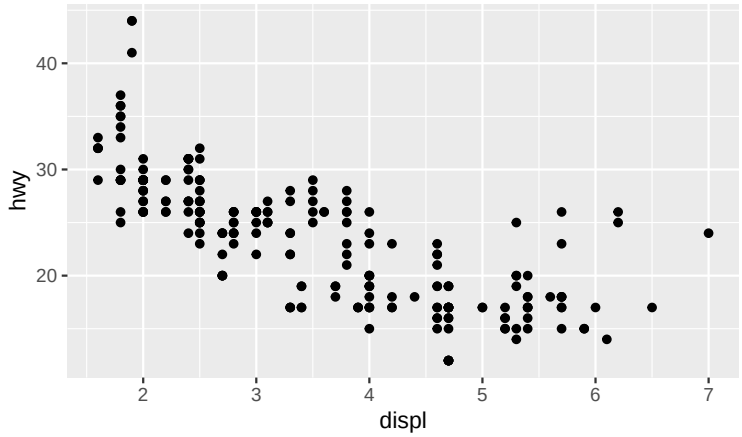
```
mpg
```

```
## # A tibble: 234 x 11
##    manufacturer model      displ  year   cyl trans      drv     cty   hwy fl    class
##    <chr>        <chr>      <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
##  1 audi         a4           1.8  1999     4 auto(l5)   f        18    29 p     compact
##  2 audi         a4           1.8  1999     4 manual(m5) f        21    29 p     compact
##  3 audi         a4           2    2008     4 manual(m6) f        20    31 p     compact
##  4 audi         a4           2    2008     4 auto(av)   f        21    30 p     compact
##  5 audi         a4           2.8  1999     6 auto(l5)   f        16    26 p     compact
##  6 audi         a4           2.8  1999     6 manual(m5) f        18    26 p     compact
##  7 audi         a4           3.1  2008     6 auto(av)   f        18    27 p     compact
##  8 audi         a4 quattro   1.8  1999     4 manual(m5) 4        18    26 p     compact
##  9 audi         a4 quattro   1.8  1999     4 auto(l5)   4        16    25 p     compact
## 10 audi         a4 quattro   2    2008     4 manual(m6) 4        20    28 p     compact
## # i 224 more rows
```

A **data frame** has variables in columns and observations in rows; `mpg` is a tibble (Lecture 5).

2.1.2 Creating a ggplot

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```



Bigger engines, lower efficiency.

- `ggplot(data = mpg)`: empty coordinate system (blank canvas) with data `mpg`.
- `geom_point()`: a **layer** of points, added with `+`.
- `mapping = aes(x = displ, y = hwy)`: variables (looked up in the data) → visual properties.

Graphing template:

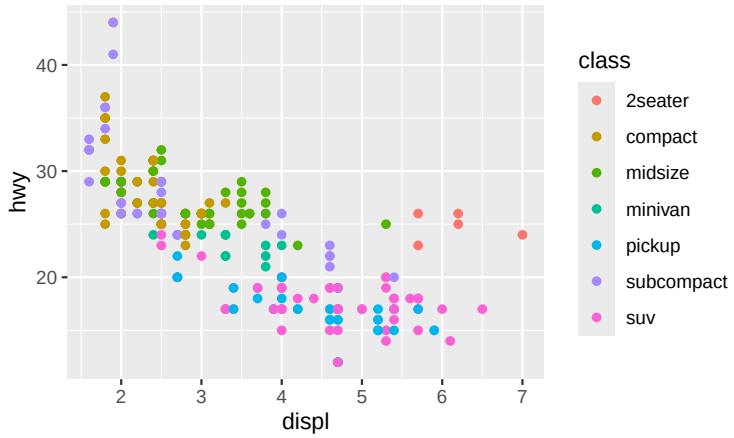
```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

2.2 Aesthetic mappings

Motivation. Aesthetics bring a third variable into a 2-D plot, e.g. to explain the large-engine, high-mileage cars.

An **aesthetic** is a visual property (position, size, shape, colour, transparency); mapping a variable to it makes it vary. Colour by `class`: the unusual cars are two-seaters.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class))
```



Scaling assigns a level to each value and adds a legend. Other aesthetics:

```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, size = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, alpha = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, shape = class))
```

- `size`, `alpha` imply an order: not for unordered `class` (ggplot2 warns).
- `shape`: at most six at a time, extra groups dropped (SUVs vanish). Shapes 0–24; 21–24 have border (`colour`) and interior (`fill`).

Setting versus mapping. A constant property goes *outside* `aes()`:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

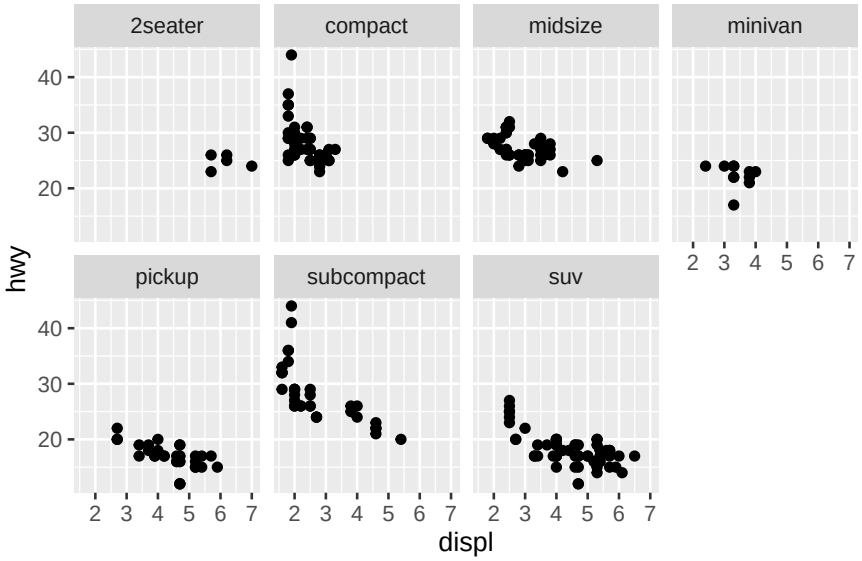
`aes(color = "blue")` would map the string `"blue"`: one group "blue", drawn red, with a legend.

2.3 Facets

Motivation. One small plot per level, side by side.

`facet_wrap()` splits by one discrete variable (a formula):

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_wrap(~ class, nrow = 2)
```



`facet_grid(drv ~ cyl)`: rows by `drv`, columns by `cyl`; `facet_grid(. ~ cyl)`: columns only. Groups are clearer, comparisons harder than with colour.

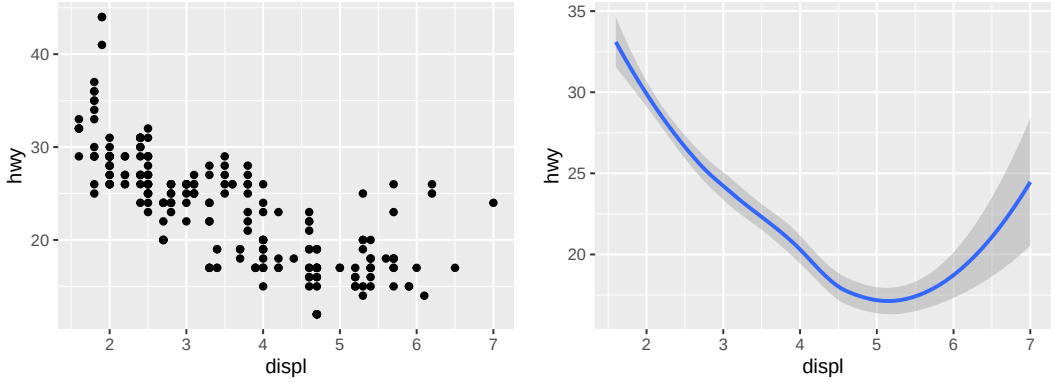
```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_grid(drv ~ class)
```

2.4 Geometric objects

Motivation. Points, curve, bars or boxes: one function, the geom.

A **geom** is the object representing the data. Same data and mapping, different geoms:

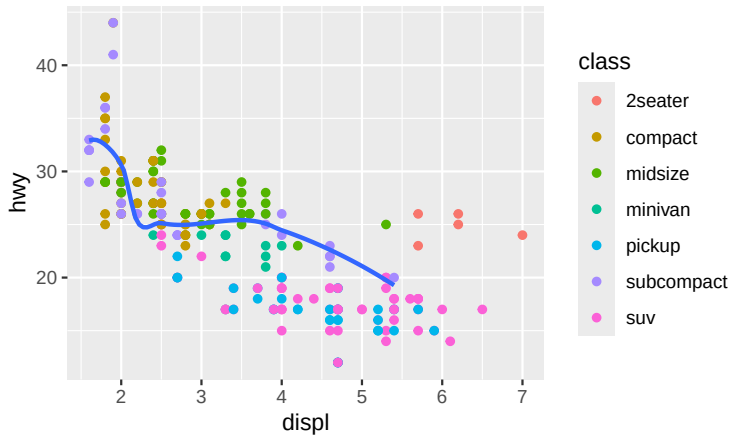
```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy))
ggplot(data = mpg) + geom_smooth(mapping = aes(x = displ, y = hwy))
```



`geom_smooth(aes(linetype = drv))` draws one curve per drive train (`linetype` fits lines, not points); mapping a discrete variable groups automatically, and `aes(group = drv)` groups without a legend.

Mappings in `ggplot()` are **global**; mappings in a geom are **local** and extend or override them. A layer can have its own data:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point(mapping = aes(color = class)) +
  geom_smooth(data = filter(mpg, class == "subcompact"), se = FALSE)
```



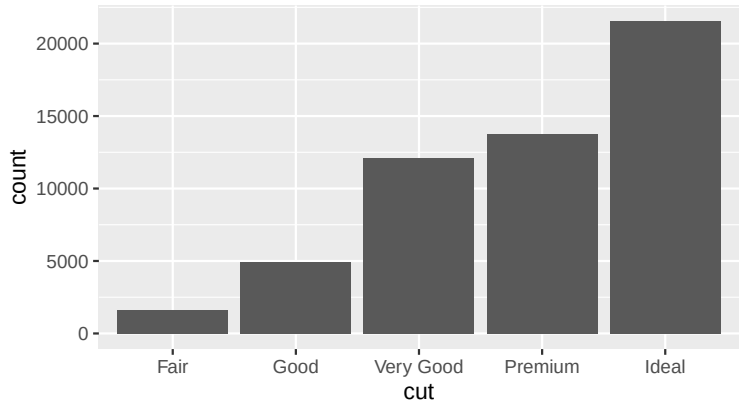
The smooth is fitted to subcompacts only (`filter()`: Lecture 3); `se = FALSE` drops the band.

2.5 Statistical transformations

Motivation. Some plots draw values computed from the data, such as counts.

`diamonds`: about 54 000 diamonds with price, carat, cut, colour, clarity.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut))
```



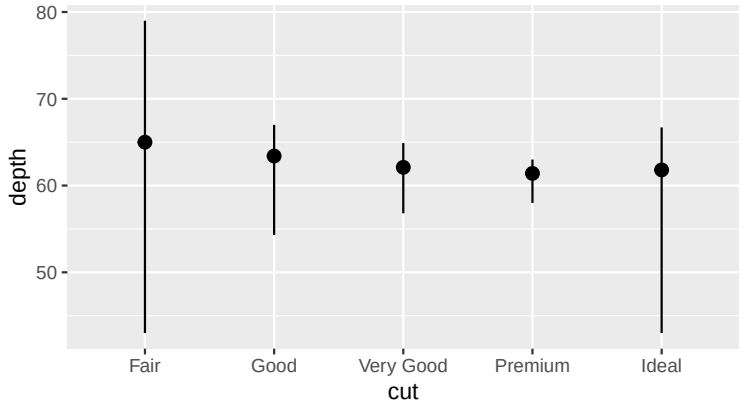
`count` is not a variable of `diamonds`. Bars, histograms, frequency polygons bin and count; smoothers fit a model; boxplots summarise. The algorithm is a **stat**: `geom_bar()` has default stat `stat_count()` and vice versa, so `ggplot(diamonds) + stat_count(aes(x = cut))` is the same plot.

Explicit stats:

- another computed variable: `aes(x = cut, y = after_stat(prop), group = 1)` (without `group = 1` every proportion is 1);

- a summary per x, e.g. median depth with range:

```
ggplot(data = diamonds) +
  stat_summary(
    mapping = aes(x = cut, y = depth),
    fun.min = min, fun.max = max, fun = median
  )
```

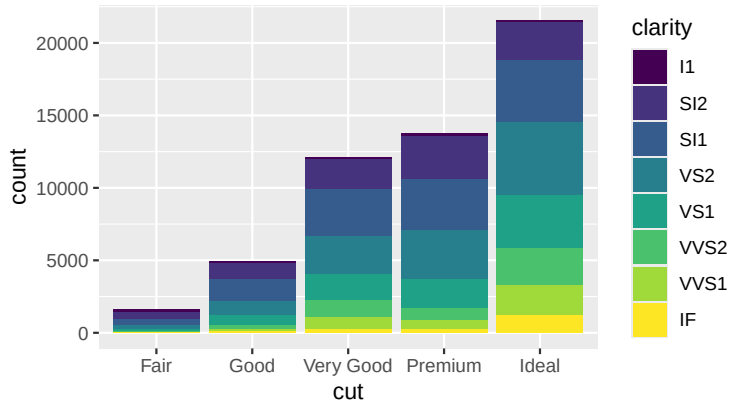


2.6 Position adjustments

Motivation. How overlapping objects are arranged changes what the plot shows.

Mapping a second variable to `fill` stacks the bars:

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity))
```



Segment heights are clarity counts within the cut, e.g. ideal:

```
diamonds %>% filter(cut == "Ideal") %>% count(clarity)
```

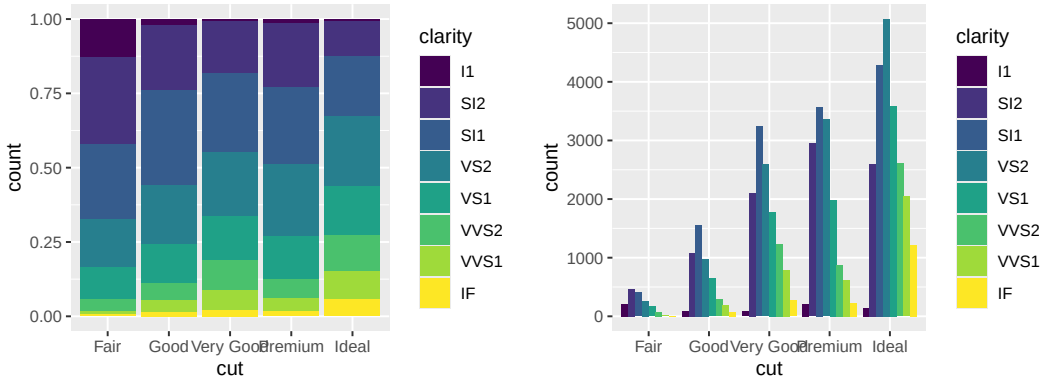
```
## # A tibble: 8 x 2
##   clarity     n
##   <ord>   <int>
## 1 I1        146
## 2 SI2      2598
## 3 SI1      4282
## 4 VS2      5071
## 5 VS1      3589
## 6 VVS2     2606
```

```
## 7 VVS1     2047
## 8 IF       1212
```

`position`:

- `"stack"` (default): segments on top of each other;
- `"identity"`: each bar from zero, overlapping (for `alpha` or lines);
- `"fill"`: stacked, rescaled to height 1: compares *proportions*;
- `"dodge"`: side by side: compares counts.

```
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "fill")
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "dodge")
```



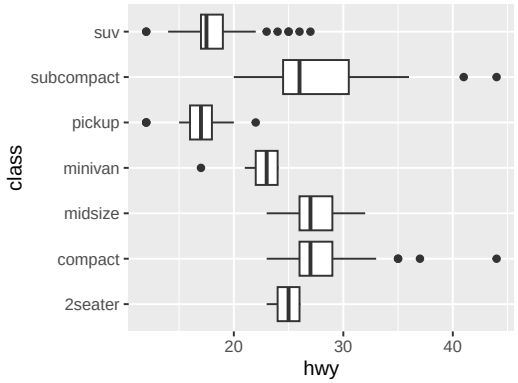
Rounded values make `mpg`'s 234 points coincide (**overplotting**). `position = "jitter"` (`geom_jitter()`) adds small noise: less accurate at small scales, more revealing at large ones.

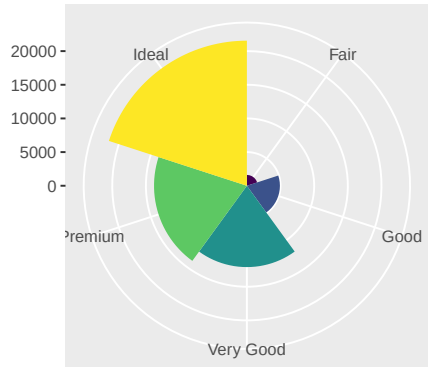
2.7 Coordinate systems

Default `coord_cartesian()` (zooms with `xlim`, `ylim`). Others:

- `coord_flip()`: swaps axes (horizontal boxplots);
- `coord_fixed(ratio = )`: fixed aspect ratio;
- `coord_quickmap()`: aspect ratio for maps (`map_data("nz")` polygons for `geom_polygon()`);
- `coord_polar()`: a bar chart becomes a Coxcomb chart.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot() + coord_flip()
bar <- ggplot(data = diamonds) +
  geom_bar(aes(x = cut, fill = cut), show.legend = FALSE, width = 1) +
  theme(aspect.ratio = 1) + labs(x = NULL, y = NULL)
bar + coord_polar()
```





2.8 The layered grammar of graphics

Motivation. Every plot fills the slots of one template.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

Seven parameters: data → stat → geom, mapping to aesthetics, position, coordinates, optional facets.

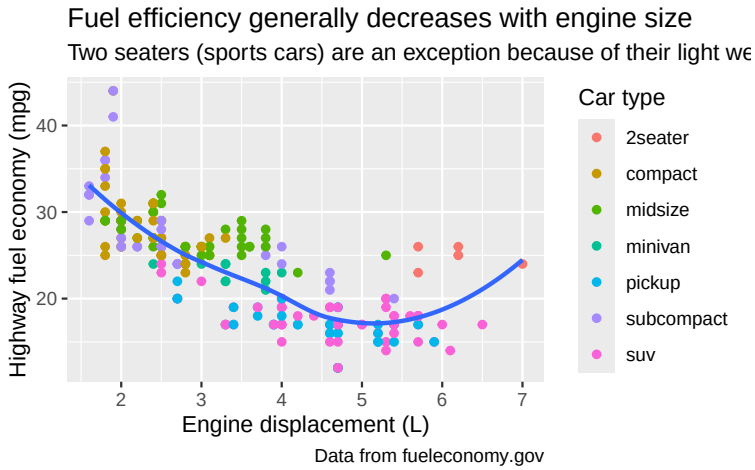
2.9 Graphics for communication

Motivation. Report plots need titles, readable labels and careful scales.

2.9.1 Labels

`labs()`: `title` (the main finding), `subtitle`, `caption` (data source), axis and legend titles:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  labs(
    title = "Fuel efficiency generally decreases with engine size",
    subtitle = "Two seaters (sports cars) are an exception because of their light weight
    ",
    caption = "Data from fueleconomy.gov",
    x = "Engine displacement (L)", y = "Highway fuel economy (mpg)", colour = "Car type"
  )
```



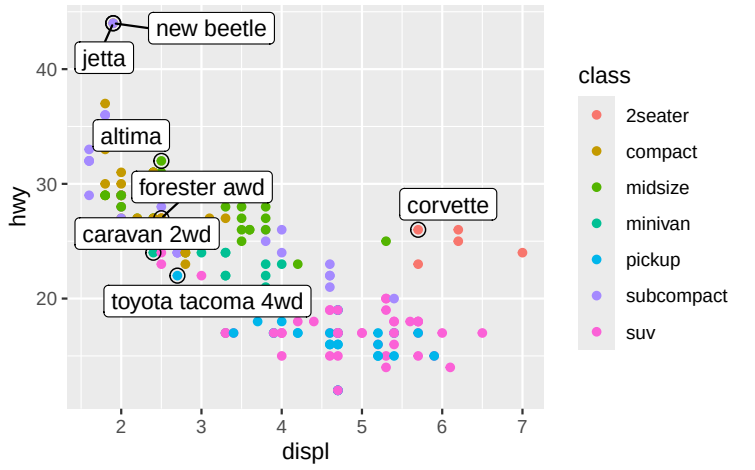
Math in labels: `quote()`, e.g. `labs(x = quote(sum(x[i]^2, i == 1, n)))`; see `?plotmath`.

2.9.2 Annotations

`geom_text()` with its own data labels observations, e.g. the most efficient car per class:

```
best_in_class <- mpg %>%
  group_by(class) %>%
  filter(row_number(desc(hwy)) == 1)
```

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(colour = class)) +
  geom_point(size = 3, shape = 1, data = best_in_class) +
  ggrepel::geom_label_repel(aes(label = model), data = best_in_class)
```



`ggrepel::geom_label_repel()` keeps labels from overlapping.

2.9.3 Scales, legends and zooming

Default scales (`scale_x_continuous()`, `scale_colour_discrete()`, ...) can be overridden to change

- the **breaks** (tick positions): `scale_y_continuous(breaks = seq(15, 40, by = 5))`;
- the **labels**: `labels = NULL` removes them;

- the **transformation**: `scale_y_log10()`, `scale_x_reverse()`.

Legend: `theme(legend.position = "bottom")` (`"left"`, `"top"`, `"right"`, `"none"`).

Zooming:

- `coord_cartesian(xlim = c(5, 7), ylim = c(10, 30))` zooms *without removing data* (smoother unchanged);
- `xlim(5, 7)`, `scale_x_continuous(limits = c(5, 7))` or filtering drop points (smoother changes).

2.9.4 Themes and saving

Eight built-in themes for non-data elements: `theme_bw()`, `theme_classic()`, `theme_minimal()`, ... `ggsave("my-plot.pdf")` saves the last plot (device size unless `width`, `height` given). Over 30 geoms, more in extensions; see the cheat sheet.